Universidade do Algarve

Faculdade de Ciências e Tecnologia

Licenciatura em Engenharia Informática

Sistemas Operativos

Trabalho Prático

Capítulo 2 - Shell

Realizado por: Diogo Carvalho nº 90247

PL: 1

Docente: Daniel José Leonardo Castro

Ano Letivo: 2025/2026

Referencias utilizadas:

Ramey, C., & Fox, B. *Bash Reference Manual*. Edition 5.3, for Bash Version 5.3. Free Software Foundation, May 2025.
MacKenzie, D., et al. *GNU Coreutils Manual*. Version 9.10. Free Software Foundation, 26 Jan. 2026.

Relatório

2.1 Primeira Parte

2.1.1 Alguns Comandos

man - abre o manual do comando (descrição, opções e exemplos). É útil para aprender rapidamente a usar um comando. Exemplo: "man ls".

info - abre a documentação “info” (GNU), em formato navegável. É útil para ler documentação mais completa (quando existe). Exemplo: "info ls".

id - mostra o UID, GID e grupos do utilizador. É útil para confirmar permissões/grupos (útil em sistemas multi-utilizador). Exemplo: "id" ou "id a90247".

whoami - imprime o nome do utilizador efetivo atual. É útil para confirmar “quem sou eu”. Exemplo: "whoami" (ex de saida: UALG\a90247).

who - lista utilizadores logados e respetivos terminais. É útil para ver quem está ligado ao sistema agora. Exemplo: "who".

tty - mostra o terminal (dispositivo) associado à tua sessão. É útil para identificar o teu terminal (ex.: pts/2). Exemplo: "tty".

finger <user> - mostra informação sobre um utilizador (se estiver disponível no sistema). É útil para obter detalhes de conta (nome, login, etc.) em sistemas onde existe. Exemplo: "finger a90247".

last - mostra histórico de logins/logouts (normalmente via /var/log/wtmp). É útil para auditoria/consulta de acessos. Exemplo: "last" ou "last -n x"(para os ultimos x).

date - mostra a data e hora atuais (e permite formatar). É útil para obter timestamp para logs/scripts. Exemplo: "date" ou date "+%Y-%m-%d %H:%M:%S"(formatado).

time <command> - mede o tempo de execução de um comando (real/user/sys). É útil para comparar performance/tempo de execução. Exemplo: "time sort big.txt > /dev/null".

passwd - altera a password (do próprio por omissão; root pode alterar outros). É útil para gerir credenciais. Exemplo: "passwd".

echo <argument> - escreve texto/variáveis no stdout. É útil para mensagens em scripts e inspeção de variáveis. Exemplo: echo "Olá"; echo "$HOME".

mesg - permite ou bloqueia mensagens no teu terminal (write/wall). É útil para controlar se outros te podem “escrever” no terminal. Exemplo: mesg n; mesg y.

mail e/ou mailx - cliente de email em modo texto (pode exigir configuração). É útil para enviar/ler email via terminal (muito usado em servidores). Exemplo: echo "corpo" | mail -s "assunto" user@dominio.

su - troca para outro utilizador (por omissão root), abrindo uma nova shell. É útil para executar ações como outro user (admin/testes). Exemplo: su -; su - joao.

bc - calculadora de linha de comandos (precisão arbitrária). É útil para fazer contas em scripts (inclui decimais). Exemplo: echo "scale=5; 10/3" | bc.

cal - mostra um calendário. É útil para consultar rapidamente meses/anos. Exemplo: cal; cal 2 2026.

clear - limpa o ecrã do terminal. É útil para “arrumar” a sessão antes de continuar. Exemplo: clear.

ps - lista processos. É útil para ver o que está a correr e consumir CPU/memória. Exemplo: ps aux; ps aux | grep firefox.

jobs - lista os jobs (tarefas) geridos pela shell atual. É útil para ver o que está em background/suspenso na tua sessão. Exemplo: jobs.

<Ctrl> <z> - suspende (STOP) o programa em foreground e devolve controlo à shell. É útil para parar temporariamente algo para depois retomar com bg/fg. Exemplo: (corres um comando) -> Ctrl+Z.

bg <pid>

- Nota importante: normalmente usa-se com JOB ID (ex.: %1), não com PID. - retoma um job suspenso em background. Exemplo: bg %1.

bg <job_id> - retoma o job indicado (ex.: %1) em background. É útil para deixar o programa a correr sem "prender" o terminal. Exemplo: bg %1.

fg <pid>

- Nota importante: normalmente usa-se com JOB ID (ex.: %1), não com PID. - traz um job para foreground. Exemplo: fg %1.

fg <job_id> - traz o job indicado para foreground. É útil para voltar a interagir com o programa no terminal. Exemplo: fg %1.

pwd - mostra o caminho do diretório atual. É útil para saber "onde estou" no sistema de ficheiros. Exemplo: pwd.

ls - lista ficheiros e diretórios. É útil para ver o conteúdo de uma pasta. Exemplo: ls; ls -lah.

cd - muda o diretório atual. É útil para navegar entre pastas. Exemplo: cd /etc; cd ..; cd.

find - procura ficheiros/diretórios recursivamente, com filtros. É útil para encontrar coisas por nome, tipo, tamanho, data, permissões, etc. Exemplo: find . -name "*.txt"; find /var/log -type f.

locate - procura por nome numa base de dados indexada (muito rápido). É útil para encontrar caminhos sem varrer o disco todo (depende de updatedb). Exemplo: locate bashrc.

Comandos de ficheiros e diretórios

pwd - mostra o caminho do diretório atual. É útil para saber "onde estou" no sistema de ficheiros. Exemplo: pwd.

ls - lista ficheiros e diretórios. É útil para ver o conteúdo de uma pasta. Exemplo: ls; ls -lah.

cd - muda o diretório atual. É útil para navegar entre pastas. Exemplo: cd /etc; cd ..; cd.

find - procura ficheiros/diretórios recursivamente, com filtros. É útil para encontrar coisas por nome, tipo, tamanho, data, permissões, etc. Exemplo: find . -name "*.txt"; find /var/log -type f.

locate - procura por nome numa base de dados indexada (muito rápido). É útil para encontrar caminhos sem varrer o disco todo (depende de updatedb). Exemplo: locate bashrc.

mkdir e rmdir

mkdir - cria diretórios. Exemplo: mkdir nova_pasta; mkdir -p a/b/c.

rmdir - remove diretórios vazios. Exemplo: rmdir pasta_vazia.

chmod - altera permissões (leitura/escrita/execução). É útil para dar execução a scripts e controlar acessos. Exemplo: chmod u+x script.sh; chmod 644 ficheiro.txt.

cat - mostra (concatena) o conteúdo de ficheiros no terminal. É útil para visualizar ficheiros pequenos. Exemplo: cat ficheiro.txt; cat a.txt b.txt > ambos.txt.

more e less

more - mostra um ficheiro “página a página” (mais simples). Exemplo: more /etc/services.

less - pager mais completo (scroll para cima/baixo, pesquisa com /). Exemplo: less /var/log/syslog.

cut - extrai colunas/campos de cada linha. É útil para “partir” linhas por delimitador (ex.: : ou ,). Exemplo: cut -d: -f1 /etc/passwd.

vi - editor de texto em terminal (vi/vim). É útil para editar ficheiros rapidamente num servidor. Exemplo: vi notas.txt.

head <filename> - mostra as primeiras linhas de um ficheiro. É útil para ver o início de logs/dados. Exemplo: head ficheiro.txt; head -n 20 ficheiro.txt.

tail <filename> - mostra as últimas linhas de um ficheiro. É útil para ver o fim de logs/dados; acompanhar logs. Exemplo: tail ficheiro.txt; tail -n 50 ficheiro.txt; tail -f ficheiro.log.

ln - cria links (hard link ou simbólico). É útil para atalhos (symlink) ou apontar para o mesmo inode (hard link). Exemplo: ln -s /etc/nginx/nginx.conf nginx.conf; ln ficheiro.txt copia_mesmo_inode.txt.

cp - copia ficheiros/diretórios. É útil para duplicar conteúdo. Exemplo: cp a.txt b.txt; cp -r pasta1/ pasta2/.

mv - move ou renomeia. É útil para reorganizar/renomear ficheiros. Exemplo: mv velho.txt novo.txt; mv ficheiro.txt /tmp/.

rm - remove ficheiros/diretórios (atenção: não vai para “reciclagem”). É útil para apagar. Exemplo: rm ficheiro.txt; rm -r pasta/; rm -i ficheiro.txt.

umask - define a máscara de permissões por omissão na criação de ficheiros/dirs. É útil para controlar permissões “default” sem ter de chmod sempre. Exemplo: umask; umask 077.

file - identifica o tipo de ficheiro pelo conteúdo. É útil para saber se é texto, binário, imagem, executável, etc. Exemplo: file programa; file imagem.png.

wc - conta linhas, palavras e bytes/caracteres. É útil para estatísticas rápidas de ficheiros. Exemplo: wc -l ficheiro.txt; wc ficheiro.txt.

split - divide um ficheiro em vários ficheiros menores. É útil para partir ficheiros grandes para envio/processamento. Exemplo: split -b 50M grande.bin parte_.

grep <expressão> <ficheiro> - procura linhas que casem com uma expressão (regex) num ficheiro. É útil para filtrar logs e procurar padrões. Exemplo: grep "ERROR" app.log; grep -n "Sistemas Operativos" notas.txt; grep -R "TODO" ..

sort - ordena linhas de texto. É útil para ordenar resultados/ficheiros (alfabético ou numérico). Exemplo: sort nomes.txt; sort -n valores.txt.

diff - compara dois ficheiros e mostra diferenças. É útil para ver alterações entre versões. Exemplo: diff a.txt b.txt; diff -u a.txt b.txt.

compress - comprime ficheiros (formato antigo. Z; pode não existir no sistema). É útil para compressão simples em sistemas que ainda suportam. Exemplo: compress ficheiro.txt -> cria ficheiro.txt.Z.

uncompress - descomprime ficheiros.Z. É útil para reverter compress. Exemplo: uncompress ficheiro.txt.Z.

chgrp - altera o grupo de um ficheiro/diretório. É útil para ajustar permissões baseadas em grupo. Exemplo: chgrp staff ficheiro.txt; chgrp -R staff pasta/.

chown - altera o dono (e opcionalmente o grupo). É útil para corrigir ownership (normalmente requer sudo). Exemplo: sudo chown joao ficheiro.txt; sudo chown joao:staff ficheiro.txt; sudo chown -R joao:staff pasta/.

df e du

df - mostra espaço livre/usado por sistema de ficheiros. Exemplo: df -h.

du - mostra espaço usado por diretórios/ficheiros. Exemplo: du -sh pasta/; du -h --max-depth=1.

mount - monta um sistema de ficheiros (disco/pen/partição) num diretório. É útil para aceder a dispositivos/partições. Exemplo: mount; sudo mount /dev/sdb1 /mnt/usb.

umount - desmonta um sistema de ficheiros. É útil para remover com segurança (evitar corrupção). Exemplo: sudo umount /mnt/usb.

2.1.2 Programação em Shell

Scripts

Executar no contexto da shell atual (carrega variáveis/funções no teu terminal):
- source teste1
- . teste1

2) Executar como programa (precisa permissões de execução):
- chmod u+x teste1
- ./teste1
(nota: usar "./" é o mais correto; só "teste1" só funciona se estiver no PATH)

2.1.2.1 Variáveis e Operadores

Variáveis (parâmetros) da Shell

Parâmetros chave (variáveis "normais")
- Nome começa por letra ou "_", seguido de alfanuméricos.
- Atribuição não leva espaços: valor=5
- Para ler: echo "$valor"

Parâmetros posicionais (argumentos do script)
- $0 -> nome do script
- $1 -> 1º argumento
- $2 -> 2º argumento
- ...

Parâmetros especiais
- representa o número de argumentos passados ao programa: $#
- referencia todos os parâmetros posicionais: $@ (ou $*)
- representa o nome do programa/script a ser executado: $0
- representa o número do processo (PID) a ser executado: $$
- representa o número do último processo executado em segundo plano: $!
- representa o estado do último comando não executado em segundo plano (exit status): $?

Comentários e shebang

- Comentário: começa com #
Ex.: # isto é um comentário

- Shebang: primeira linha do script a indicar o interpretador:
#!/bin/bash

Operadores

Lógicos
- AND: && (em testes também existe -a)
- OR: || (em testes também existe -o)
- NOT: ! (nega condição/comando)

Relacionais
- menor do que: -lt
- menor ou igual a: -le
- igual a: -eq
- maior do que: -gt
- diferente: -ne

Strings
- iguais: =
- diferentes: !=
- comprimento maior que zero: -n
- comprimento igual a zero: -z

Ficheiros
- é um directório: -d
- é um ficheiro regular: -f
- tem permissão de leitura: -r
- tem permissão de escrita: -w
- tem permissão de execução: -x
- tem conteúdo (tamanho > 0): -s
- pertence ao utilizador actual: -O
- pertence ao grupo actual: -G
- é um pipe: -p
- mais recente (ficheiro1 mais novo que 2): -nt
- mais antigo (ficheiro1 mais velho que 2): -ot

2.1.2.2 Estruturas de controlo de execução em “bash”

if

Sintaxe:
if <comando-ou-teste>; then
...
elif <comando-ou-teste>; then
...
else
...
fi

Exemplo:
if [-f "a.txt"]; then
echo "Existe"
else
echo "Não existe"
fi

for

Sintaxe (lista):
for v in item1 item2 item3; do
...
done

Exemplo:
for d in /bin /etc; do
echo "Dir: $d"
done

case

Sintaxe:

```
case "$var" in
padrao1) ... ;;
padrao2) ... ;;
*) ... ;;
esac
```

Exemplo:

```
case "$1" in
1) date ;;
2) date "+%Y-%m-%d %H:%M:%S" ;;
3) exit 0 ;;
*) echo "opções válidas [1 2 3]" ;;
esac
```

while

Sintaxe:

```
while <teste>; do
...
done
```

Exemplo:

```
i=1
while [ "$i" -le 5 ]; do
echo "$i"
i=$((i+1))
done
```

until

Sintaxe:

```
until <teste>; do
...
done
```

Exemplo:

```
i=1
until [ "$i" -gt 5 ]; do
echo "$i"
i=$((i+1))
done
```

2.1.2.3 Exemplos

1) exif1.bash

```
#!/bin/bash
# exif1 - verifica se um diretório existe

if [ $# -ne 1 ]; then
  echo "Uso: $0 <diretorio>"
  exit 1
fi

DIR="$1"

if [ -d "$DIR" ]; then
  echo "O diretório existe: $DIR"
else
  echo "O diretório NÃO existe: $DIR"
fi
```

2) exif2.bash

```
#!/bin/bash

# exif2 - verifica se ficheiro existe e tem conteúdo; senão cria a
partir do stdin até Ctrl+C


if [ $# -ne 1 ]; then

  echo "Uso: $0 <ficheiro>"

  exit 1

fi


FILE="$1"


if [ -f "$FILE" ] && [ -s "$FILE" ]; then

  echo "OK: o ficheiro existe e tem conteúdo: $FILE"

  exit 0

fi

echo "O ficheiro não existe ou está vazio. Vou criar/recriar: $FILE"

echo "Escreve linhas (Ctrl+C para terminar)."


# cria/limpa o ficheiro

: > "$FILE"


# lê do stdin e acrescenta ao ficheiro até Ctrl+C

while read line; do

  echo "$line" >> "$FILE"

done
```

3) excase1.bash

```
#!/bin/bash

# excase1 - menu com case


echo "1) Data do sistema"

echo "2) Data e hora do sistema"

echo "3) Sair"

echo -n "Opção: "

read opt


case "$opt" in

  1)

    date +%F

    ;;

  2)

    date

    ;;

  3)

    exit 0

    ;;

  *)

    echo "opções válidas [1 2 3]"

    exit 1

    ;;

esac
```

4) exfor1.bash

```
#!/bin/bash

# exfor1 - entra em /bin e /etc por 2 segundos, lista, e volta ao
diretório inicial


INI="$(pwd)"


for d in /bin /etc; do

  if [ -d "$d" ]; then

    cd "$d"

    echo "== Diretório atual: $(pwd) =="

    ls

    sleep 2

  else

    echo "Aviso: diretório não existe: $d"

  fi

done


cd "$INI"

echo "Voltei ao diretório inicial: $(pwd)"
```

5) exwhile1.bash

```
#!/bin/bash

# exwhile1 - contador 1..20 com while


i=1

while [ $i -le 20 ]; do

  echo "$i"

  i=$((i + 1))

done
```

6) exuntil1.bash

```
#!/bin/bash

# exuntil1 - contador 1..20 com until


i=1

until [ $i -gt 20 ]; do

  echo "$i"

  i=$((i + 1))

done
```

7) soma.bash

```
#!/bin/bash

# soma - função soma de dois inteiros + validação do n° de argumentos


soma() {

  echo $(( $1 + $2 ))

}
```

```
if [ $# -ne 2 ]; then
  echo "Uso: $0 <int1> <int2>"
  exit 1
fi

# validação simples: tenta avaliar como aritmética
a="$1"
b="$2"

# Se não forem números inteiros, o bash costuma dar erro em aritmética;
# fazemos um teste indireto:
tmp=$((a + 0)) 2>/dev/null
if [ $? -ne 0 ]; then
  echo "Erro: o 1° argumento não é inteiro."
  exit 1
fi

tmp=$((b + 0)) 2>/dev/null
if [ $? -ne 0 ]; then
  echo "Erro: o 2° argumento não é inteiro."
  exit 1
fi

res="$(soma "$a" "$b")"
echo "$a + $b = $res"
```

2.2 Segunda Parte: Exercícios Propostos

1) exfor2.bash

VERSÃO INICIAL (base 1..20)

```
#!/bin/bash
# exfor2 - Fase 1: contador 1..20

for ((i=1; i<=20; i=i+1)); do
echo "$i"
done
```

VERSÃO FINAL (só ímpares 1..20)

```
#!/bin/bash

# exfor2 - Fase 2: pede limites, valida, mostra só ímpares


is_int() {
  case "$1" in
    ""|*[!0-9-]*|- ) return 1 ;;
    -*)
      [ "$1" = "-" ] && return 1
      case "${1#-}" in
        ""|*[!0-9]*) return 1 ;;
        *) return 0 ;;
      esac
      ;;
    *) return 0 ;;
  esac
}
```

```
echo -n "Limite inferior: "
read low
echo -n "Limite superior: "
read high

if ! is_int "$low"; then
  echo "Erro: limite inferior não é inteiro."
  exit 1
fi

if ! is_int "$high"; then
  echo "Erro: limite superior não é inteiro."
  exit 1
fi

if [ "$low" -gt "$high" ]; then
  echo "Erro: limite inferior é maior que o limite superior."
  exit 1
fi

for ((i=low; i<=high; i=i+1)); do
  if [ $((i % 2)) -ne 0 ]; then
    echo "$i"
  fi
done
```

2) dobro.bash

```
--------------------------------------------------
#!/bin/bash
# dobro - função dobro com validação de argumento numérico (inteiro)

is_int() {
  case "$1" in
    ""|*[!0-9-]*|- ) return 1 ;;
    -*)
      [ "$1" = "-" ] && return 1
      case "${1#-}" in
        ""|*[!0-9]*) return 1 ;;
        *) return 0 ;;
      esac
      ;;
    *) return 0 ;;
  esac
}

dobro() {
  if [ $# -ne 1 ]; then
    echo "Erro: a função dobro precisa de 1 argumento." 1>&2
    return 1
  fi
  if ! is_int "$1"; then
    echo "Erro: argumento não é inteiro." 1>&2
    return 1
  fi
```

```
  echo $((2 * $1))
}

if [ $# -ne 1 ]; then
  echo "Uso: $0 <inteiro>"
  exit 1
fi

res="$(dobro "$1")"
if [ $? -ne 0 ]; then
  exit 1
fi

echo "Dobro de $1 = $res"
```

3) fatorial.bash

```
#!/bin/bash

is_int() { [[ "$1" =~ ^[0-9]+$ ]]; }

fact_iter() {
  local n="$1"
  local acc=1
  local i
  for ((i=2; i<=n; i++)); do
    acc=$((acc * i))
  done
  echo "$acc"
}

fact_rec() {
  local n="$1"
  if [ "$n" -le 1 ]; then
    echo 1
  else
    echo $(( n * $(fact_rec $((n-1))) ))
  fi
}

if [ $# -ne 1 ]; then
  echo "Uso: $0 <n>=0"
  exit 1
```

```
fi


if ! is_int "$1"; then
  echo "Erro: n tem de ser um inteiro >= 0."
  exit 1
fi


n="$1"
echo "Factorial iterativo de $n: $(fact_iter "$n")"
echo "Factorial recursivo de $n: $(fact_rec  "$n")"
```

4) Script para analisar last -f /var/log/wtmp (parse_last_wtmp.bash)

Objetivo: por utilizador -> nº de logins + tempo total.

Se ainda não saiu, conta até agora.

```
#!/bin/bash
# parse_last_wtmp.bash
# Lê teste.wtmp (saída do last -f /var/log/wtmp > teste.wtmp)
# Para cada utilizador: n° de entradas e tempo total de utilização.
# Se "still logged in", conta até agora.

FILE="teste.wtmp"
if [ $# -ge 1 ]; then
  FILE="$1"
fi

if [ ! -f "$FILE" ]; then
  echo "Erro: ficheiro não existe: $FILE"
  exit 1
fi

declare -A count
declare -A seconds

# Converte "HH:MM" ou "D+HH:MM" em segundos
dur_to_seconds() {
  s="$1"
  days=0
```

```
  case "$s" in
    *+*)
      days="${s%%+*}"
      rest="${s#*+}"
      ;;
    *)
      rest="$s"
      ;;
  esac

  hh="${rest%%:*}"
  mm="${rest#*:}"

  # FORÇAR BASE 10 (evita "08"/"09" serem interpretados como octal)
  days=$((10#$days))
  hh=$((10#$hh))
  mm=$((10#$mm))

  echo $((days*86400 + hh*3600 + mm*60))
}

# Devolve a posição (índice 1-based) do token do mês (Jan..Dec) na
linha
month_index() {
  i=1
  for tok in "$@"; do
    case "$tok" in
      Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)
        echo "$i"
```

```
        return 0
        ;;
    esac
    i=$((i + 1))
  done
  echo 0
  return 1
}

now_epoch="$(date +%s)"
year="$(date +%Y)"

while IFS= read -r line; do
  [ -z "$line" ] && continue

  # Ignorar linhas de cabeçalho/rodapé
  case "$line" in
    wtmp\ begins*|btmp\ begins*|reboot*|shutdown*) continue ;;
  esac

  set -- $line
  user="$1"
  [ -z "$user" ] && continue

  # Contar entradas
  count["$user"]=$(( ${count["$user"]:-0} + 1 ))

  # Token final
```

```
lasttok="${!#}"

# Caso com duração no fim: "(HH:MM)" ou "(D+HH:MM)"
case "$lasttok" in
  \(*\))
    d="${lasttok#(}"
    d="${d%)}"
    add="$(dur_to_seconds "$d")"
    seconds["$user"]=$(( ${seconds["$user"]:-0} + add ))
    continue
    ;;
esac

# Caso "still logged in" (sem "(...)"): calcular agora - login_time
mi="$(month_index "$@")"
if [ "$mi" -eq 0 ]; then
  # formato inesperado; ignora
  continue
fi

# tokens: ... <Mon> <Day> <HH:MM> ...
eval mon="\${$mi}"
eval day="\${$((mi+1))}"
eval hm="\${$((mi+2))}"

login_epoch="$(date -d "$year $mon $day $hm" +%s 2>/dev/null)"
if [ -z "$login_epoch" ]; then
  continue
```

```
  fi

  add=$(( now_epoch - login_epoch ))
  [ "$add" -lt 0 ] && add=0
  seconds["$user"]=$(( ${seconds["$user"]:-0} + add ))

done < "$FILE"

echo "Utilizador | Entradas | Tempo total"
echo "-----------------------------------"

for u in "${!count[@]}"; do
  total="${seconds["$u"]:-0}"
  hh=$(( total / 3600 ))
  mm=$(( (total % 3600) / 60 ))
  ss=$(( total % 60 ))
  printf "%-9s | %-8s | %02d:%02d:%02d\n" "$u" "${count["$u"]}" "$hh"
"$mm" "$ss"
done
```

2.3 Terceira Parte: Questões gerais e exercícios adicionais

1) O que é uma shell?

Uma shell é um interpretador de comandos que funciona como interface entre o utilizador e o sistema operativo. A shell lê comandos (introduzidos no terminal ou num script), interpreta-os e executa-os, podendo também gerir aspetos como variáveis, parâmetros, redireccionamentos e estruturas de controlo (if, for, while, case, etc.).

2) Identifique três programas de shell.

Três exemplos comuns são sh, bash, csh.

3) Por omissão, há três ficheiros standard sempre abertos. Quais e qual o seu objectivo?

Por convenção, qualquer processo tem três streams standard abertas:

Stdin: entrada standard, normalmente o teclado/terminal.

Stdout: saída standard, normalmente o ecrã/terminal.

Stderr: saída de erro, normalmente o ecrã/terminal, separada de stdout para permitir redirecionar erros independentemente.

4) Quando é que se classifica um comando como simples, encadeado ou composto?

Comando simples é uma execução direta de um programa (com argumentos), eventualmente com redirecionamentos.

Exemplos são: ls, date, ls > out.txt.

Comando encadeado é uma sequência/ligação de comandos através de operadores da shell.

Exemplos são: ";", "&&", "||" e "|".

Comando composto são comandos construídos com estruturas próprias da shell, como:

if ... fi, for ... do ... done, while ... do ... done, until ... do ... done, case ... esac.

5) Se numa sessão quiser mudar de shell, como o pode fazer?

Basta executar a shell pretendida no terminal, utilizando:

bash

sh

Isso inicia uma nova shell "dentro" da sessão atual. Para regressar à anterior, termina-se a shell atual com exit (ou Ctrl+D).

6) Sabendo que em /usr/bin coexistem executáveis e scripts, quantos são os scripts?

Em /usr/bin coexistem scripts e programas executáveis. Após realizar a contagem no sistema obtive 417 Scripts, 795 Executáveis e 440 Outros ficheiros que são ligações(links).

7) Script testa_entradas.sh: avaliar se a chamada foi feita com argumentos.

```
#!/bin/bash

# testa_entradas.sh - verifica se o script foi chamado com argumentos


if [ $# -eq 0 ]; then

  echo "Erro: não foram fornecidos argumentos."

  echo "Uso: $0 <arg1> [arg2 ...]"

  exit 1

fi


echo "OK: foram fornecidos $# argumento(s)."
```

8) Scripts em bash

8.1) Listar ficheiros com uma extensão definida pelo utilizador

```
#!/bin/bash

# lista_extensao.sh - lista ficheiros com extensão indicada


if [ $# -ne 1 ]; then

  echo "Uso: $0 <extensao>"

  exit 1

fi


ext="$1"

# permitir que o utilizador escreva ".txt" ou "txt"

case "$ext" in

  .*) ext="${ext#.}" ;;

esac


for f in *; do

  [ -f "$f" ] || continue

  case "$f" in

    *."$ext") echo "$f" ;;

  esac

done
```

8.2) Identificar diretórios com tamanho > limiar e comprimi-los

```
#!/bin/bash
# compacta_dir_grandes.sh - comprime diretórios com tamanho > limiar (KB)

if [ $# -ne 1 ]; then
  echo "Uso: $0 <limiar_KB>"
  exit 1
fi

lim="$1"
case "$lim" in
  ""|*[!0-9]*) echo "Erro: limiar tem de ser inteiro (KB)."; exit 1 ;;
esac

for d in *; do
  [ -d "$d" ] || continue

  # du -sk devolve: "<KB> <nome>"
  set -- $(du -sk "$d" 2>/dev/null)
  size="$1"
  case "$size" in
    ""|*[!0-9]*) continue ;;
  esac
  if [ "$size" -gt "$lim" ]; then
    echo "A comprimir: $d (tamanho ${size}KB)"
    tar -czf "${d}.tar.gz" "$d"
  fi
done
```

8.3) Listar ficheiros cuja data esteja num intervalo definido pelo utilizador

```
#!/bin/bash

# lista_por_data.sh - lista ficheiros (neste diretório) com data entre
duas datas

# Datas no formato: YYYY-MM-DD


echo -n "Data inicial (YYYY-MM-DD): "

read ini

echo -n "Data final   (YYYY-MM-DD): "

read fim


# validar minimamente: tem de ter 10 chars e 2 hífens

case "$ini" in

  ????-??-??) ;;

  *) echo "Erro: data inicial inválida."; exit 1 ;;

esac

case "$fim" in

  ????-??-??) ;;

  *) echo "Erro: data final inválida."; exit 1 ;;

esac


fim_next="$(date -d "$fim + 1 day" +%Y-%m-%d 2>/dev/null)"

if [ -z "$fim_next" ]; then

  echo "Erro: data final inválida."

  exit 1

fi

# Ficheiros regulares apenas no diretório atual

find . -maxdepth 1 -type f -newermt "$ini" ! -newermt "$fim_next" -
print
```

8.4) Listar o login e nome dos utilizadores do sistema

```
#!/bin/bash

# lista_users.sh - mostra login e nome a partir de /etc/passwd


while IFS=: read -r login x uid gid gecos home shell; do

  nome="$gecos"

  case "$nome" in

    *,*) nome="${nome%%,*}" ;;

  esac

  printf "%-20s %s\n" "$login" "$nome"

done < /etc/passwd
```

8.5) Listar os ficheiros que contenham uma dada expressão como,

por exemplo “Sistemas Operativos”

```
#!/bin/bash

# procura_expressao.sh - lista ficheiros que contêm a expressão
(procura recursiva)

if [ $# -lt 1 ]; then

  echo "Uso: $0 <expressao>"

  exit 1

fi


expr="$1"

shift

for w in "$@"; do

  expr="$expr $w"

done

grep -R -l "$expr" .
```